

UNIVERSITAT POLITÈCNICA DE CATALUNYA

Escola Tècnica Superior d'Enginyeria de Telecomunicacions de Barcelona (ETSETB)

Practice 1

Grau en Enginyeria Física

Authors: [Author Name]

Subject: SIMCOM

Date: [Date]

1 Exercise 1

1.1 Objective

[State the problem and the intended goal]

1.2 Code

```
1 PROGRAM EX1
2   IMPLICIT NONE
3   INTEGER :: A
4   REAL    :: B, RESULT
5   INCLUDE './../chdir_to_code.inc'
6
7   PRINT *, 'Enter an integer:'
8   READ(*, *) A
9
10  PRINT *, 'Enter a real number:'
11  READ(*, *) B
12
13  RESULT = REAL(A) + B
14
15  PRINT '(A, F10.2)', 'The sum is: ', RESULT
16
17  OPEN(UNIT=10, FILE='ex1_results.txt', STATUS='REPLACE', ACTION='WRITE')
18  WRITE(10, '(A, F10.2)') 'The sum is: ', RESULT
19  CLOSE(10)
20
21 END PROGRAM EX1
```

1.3 Results

```
1 The sum is:      3.30
```

1.4 Discussion & Notes

[Comment on critical or non-trivial components]

2 Exercise 2

2.1 Objective

[State the problem and the intended goal]

2.2 Code

```
1 PROGRAM EX2
2   IMPLICIT NONE
3   DOUBLE PRECISION, PARAMETER :: pi = 3.14159265358979323846D0
4   DOUBLE PRECISION :: pi_aprox = 0.0D0, error = 1.0D0
5   INTEGER :: K = 0, sign_K = 1
6   INCLUDE '../chdir_to_code.inc'
7   OPEN(UNIT=10, FILE='ex2_results.dat', STATUS='REPLACE', ACTION='WRITE')
8
9   DO WHILE (ABS(error) > 1.0D-6)
10      pi_aprox = pi_aprox + sign_K * 4.0D0 / (2.0D0 * DBLE(K) + 1.0D0)
11      error = pi_aprox - pi
12
13      K = K + 1
14      WRITE(10, '(I8, 1X, ES16.8)') K, pi_aprox
15      sign_K = -sign_K
16   END DO
17
18   CLOSE(10)
19
20   PRINT *, 'The approximate value of pi for K =', K, 'is: ', pi_aprox
21 END PROGRAM EX2
```

2.3 Results

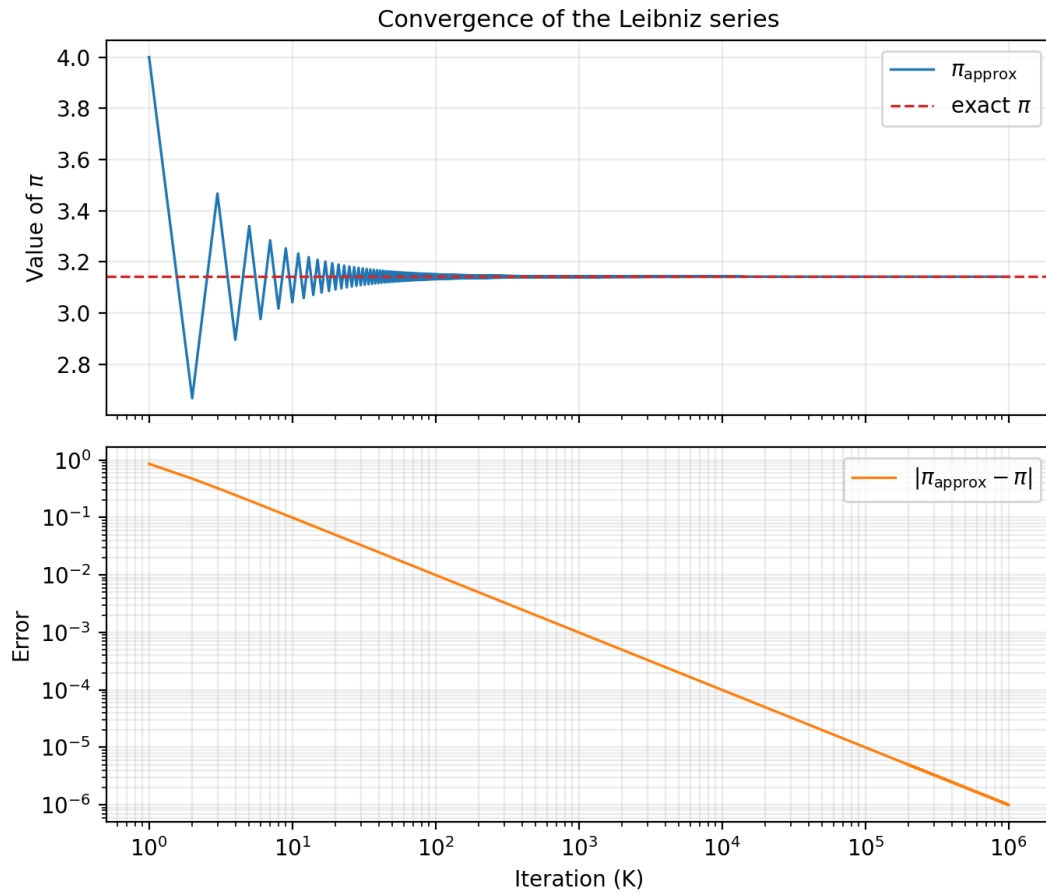


Figure 1: Output of exercise 2

2.4 Discussion & Notes

[Comment on critical or non-trivial components]

3 Exercise 3

3.1 Objective

[State the problem and the intended goal]

3.2 Code

```
1 PROGRAM EX3
2   IMPLICIT NONE
3   INTEGER, PARAMETER :: npts = 2001
4   INTEGER, PARAMETER :: wmax = 500
5   REAL, DIMENSION(npts) :: C, t
6   REAL, DIMENSION(0:wmax) :: result
7   INTEGER :: i, w
8   INCLUDE '../chdir_to_code.inc'
9
10  OPEN(UNIT=10, FILE='ex3_data.dat', STATUS='OLD', ACTION='READ')
11  DO i = 1, npts
12    READ(10, *) t(i), C(i)
13  END DO
14  CLOSE(10)
15
16  CALL simpson(C, t, result)
17
18  OPEN(UNIT=20, FILE='ex3_results.dat', STATUS='REPLACE', ACTION='WRITE')
19  DO w = 0, wmax
20    WRITE(20, '(I8, 1X, ES16.8)') w, result(w)
21  END DO
22  CLOSE(20)
23
24  PRINT *, 'Results written to ex3_results.dat'
25  PRINT *, 'F(w=0) = ', result(0)
26
27  CONTAINS
28
29  SUBROUTINE simpson(C, t, result)
30    IMPLICIT NONE
31    REAL, DIMENSION(:), INTENT(IN) :: C, t
32    REAL, DIMENSION(0:), INTENT(OUT) :: result
33    INTEGER :: i, w, n
34    REAL :: h, s
35    REAL, ALLOCATABLE :: f(:)
36
37    n = SIZE(t)
38    h = t(2) - t(1)
39    ALLOCATE(f(n))
40
41    DO w = 0, 500
42      DO i = 1, n
43        f(i) = C(i) * COS(REAL(w) * t(i))
44      END DO
45
46      ! Composite Simpson (even number of intervals, step 2)
47      s = 0.0
48      DO i = 2, n - 1, 2
49        s = s + h / 3.0 * (f(i-1) + 4.0 * f(i) + f(i+1))
50      END DO
51      result(w) = s
52    END DO
53
54    DEALLOCATE(f)
```

```

55  END SUBROUTINE simpson
56
57  END PROGRAM EX3

```

3.3 Results

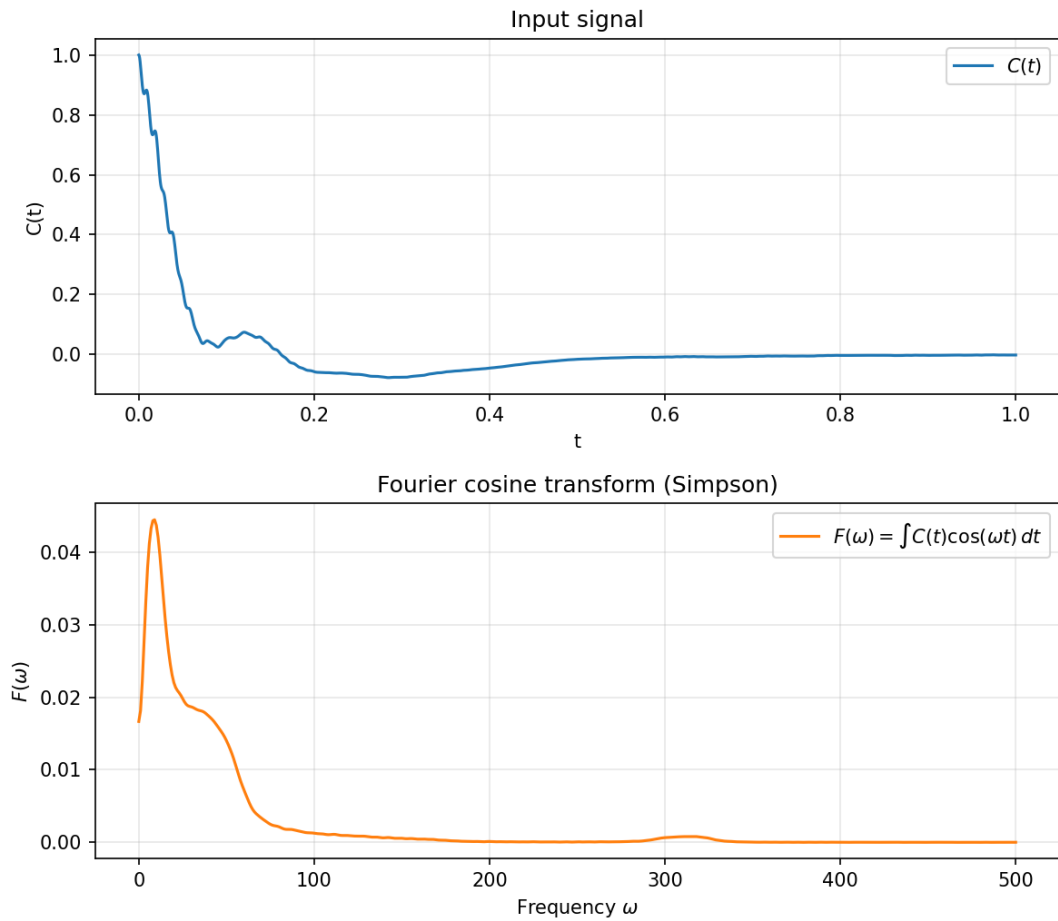


Figure 2: Output of exercise 3

3.4 Discussion & Notes

[Comment on critical or non-trivial components]

4 Exercise 4

4.1 Objective

[State the problem and the intended goal]

4.2 Code

```
1 PROGRAM EX4
2   IMPLICIT NONE
3   INTEGER, PARAMETER :: nmax = 1000
4   INTEGER, PARAMETER :: nruns = 3
5   REAL, DIMENSION(nmax) :: x, y
6   REAL :: A, A_average, A_exact, A_deviation, A_deviation_average
7   INTEGER :: N, J
8   INCLUDE '../chdir_to_code.inc'
9
10  A_exact = 4.0 * (1.0 + LOG(11.0))
11
12  OPEN(UNIT=10, FILE='ex4_results.dat', STATUS='REPLACE', ACTION='WRITE')
13  DO N = 100, nmax, 100
14    A_average = 0.0
15    A_deviation = 0.0
16    DO J = 1, nruns
17      CALL RANDOM_NUMBER(x(1:N))
18      CALL RANDOM_NUMBER(y(1:N))
19      x(1:N) = x(1:N) * 2.0 - 1.0
20      y(1:N) = y(1:N) * 22.0 - 11.0
21      CALL calculate_area(x, y, N, A)
22      A_average = A_average + A
23      A_deviation = A_deviation + (A - A_exact)**2
24    END DO
25    A_average = A_average / REAL(nruns)
26    A_deviation_average = SQRT(A_deviation / REAL(nruns))
27    WRITE(10, '(I8, 1X, ES16.8, 1X, ES16.8)') N, A_average, A_deviation_average
28  END DO
29  CLOSE(10)
30
31  PRINT *, 'Results written to ex4_results.dat'
32  PRINT *, 'Exact area = ', A_exact
33
34  CONTAINS
35
36  SUBROUTINE calculate_area(x, y, N, A)
37    IMPLICIT NONE
38    INTEGER, INTENT(IN) :: N
39    REAL, DIMENSION(:), INTENT(IN) :: x, y
40    REAL, INTENT(OUT) :: A
41    INTEGER :: k, Nh
42
43    Nh = 0
44    DO k = 1, N
45      IF (x(k) /= 0.0) THEN
46        IF (ABS(y(k)) <= ABS(1.0 / x(k))) THEN
47          Nh = Nh + 1
48        END IF
49      END IF
50    END DO
51    A = 44.0 * REAL(Nh) / REAL(N)
52  END SUBROUTINE calculate_area
53
54  END PROGRAM EX4
```

4.3 Results

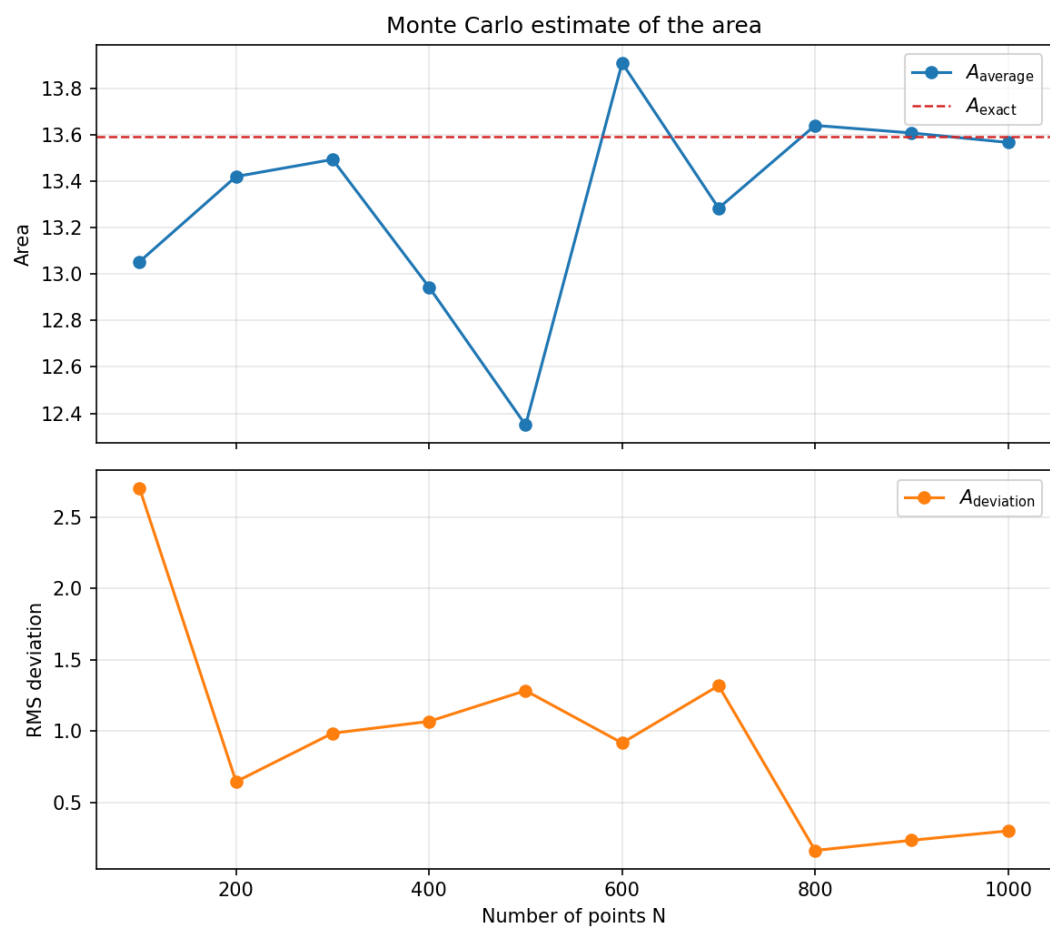


Figure 3: Output of exercise 4

4.4 Discussion & Notes

[Comment on critical or non-trivial components]

5 Exercise 5

5.1 Objective

[State the problem and the intended goal]

5.2 Code

```
1 PROGRAM EX5
2   IMPLICIT NONE
3   REAL :: m=0.200,k=2.0
4   REAL :: t_max=6.0, dt, x_0=0.05, v_0=0.0, t_0=0.0
5   REAL, ALLOCATABLE :: x_euler_001(:),v_euler_001(:), t_euler_001(:)
6   REAL, ALLOCATABLE :: x_euler_01(:),v_euler_01(:), t_euler_01(:)
7   REAL, ALLOCATABLE :: x_euler_02(:),v_euler_02(:), t_euler_02(:)
8
9   REAL, ALLOCATABLE :: x_euler_predictor(:),v_euler_predictor(:), t_euler_predictor(:)
10  REAL, ALLOCATABLE :: x_verlet(:),v_verlet(:), t_verlet(:)
11  INTEGER :: i
12
13  dt=0.001
14  CALL EULER(m,k,t_max,dt,x_0,v_0,t_0,x_euler_001,v_euler_001,t_euler_001)
15  OPEN(10,file='euler_001.dat')
16  WRITE(10,*) 't,x,v'
17  DO i = 1, NINT(t_max/dt)+1
18    WRITE(10,*) t_euler_001(i),x_euler_001(i),v_euler_001(i)
19  END DO
20  CLOSE(10)
21
22  dt=0.01
23  CALL EULER(m,k,t_max,dt,x_0,v_0,t_0,x_euler_01,v_euler_01,t_euler_01)
24  OPEN(11,file='euler_01.dat')
25  WRITE(11,*) 't,x,v'
26  DO i = 1, NINT(t_max/dt)+1
27    WRITE(11,*) t_euler_01(i),x_euler_01(i),v_euler_01(i)
28  END DO
29  CLOSE(11)
30
31  dt=0.02
32  CALL EULER(m,k,t_max,dt,x_0,v_0,t_0,x_euler_02,v_euler_02,t_euler_02)
33  OPEN(12,file='euler_02.dat')
34  WRITE(12,*) 't,x,v'
35  DO i = 1, NINT(t_max/dt)+1
36    WRITE(12,*) t_euler_02(i),x_euler_02(i),v_euler_02(i)
37  END DO
38  CLOSE(12)
39
40  CALL EULER_PREDICTOR(m,k,t_max,dt,x_0,v_0,t_0,x_euler_predictor,v_euler_predictor,
41    t_euler_predictor)
42  OPEN(13,file='euler_predictor.dat')
43  WRITE(13,*) 't,x,v'
44  DO i = 1, NINT(t_max/dt)+1
45    WRITE(13,*) t_euler_predictor(i),x_euler_predictor(i),v_euler_predictor(i)
46  END DO
47  CLOSE(13)
48
49  CALL VERLET(m,k,t_max,dt,x_0,v_0,t_0,x_verlet,v_verlet,t_verlet)
50  OPEN(14,file='verlet.dat')
51  WRITE(14,*) 't,x,v'
52  DO i = 1, NINT(t_max/dt)+1
53    WRITE(14,*) t_verlet(i),x_verlet(i),v_verlet(i)
54  END DO
```

```

54     CLOSE(14)
55
56     CONTAINS
57     SUBROUTINE EULER(m,k,t_max,dt,x_0,v_0,t_0,x,v,t)
58         IMPLICIT NONE
59         REAL, INTENT(IN) :: m,k,t_max,dt,x_0,v_0,t_0
60         REAL, ALLOCATABLE, DIMENSION(:), INTENT(OUT) :: x,v,t
61         REAL :: a
62         INTEGER :: i,n
63
64         n = NINT((t_max - t_0) / dt) + 1
65         ALLOCATE(x(n), v(n), t(n))
66
67         x(1)=x_0
68         v(1)=v_0
69         t(1)=t_0
70
71         DO i = 2, n
72             a=-(k/m)*x(i-1)
73             v(i)=v(i-1)+a*dt
74             x(i)=x(i-1)+v(i-1)*dt+0.5*a*dt**2
75             t(i)=t(i-1)+dt
76         END DO
77     END SUBROUTINE EULER
78
79     SUBROUTINE VERLET(m, k, t_max, dt, x_0, v_0, t_0, x, v, t)
80         IMPLICIT NONE
81         REAL, INTENT(IN) :: m, k, t_max, dt, x_0, v_0, t_0
82         REAL, ALLOCATABLE, DIMENSION(:), INTENT(OUT) :: x, v, t
83         REAL :: a
84         INTEGER :: i, n
85
86         n = NINT((t_max - t_0) / dt) + 1
87         ALLOCATE(x(n), v(n), t(n))
88
89         t(1) = t_0
90         x(1) = x_0
91         v(1) = v_0
92         a = -(k / m) * x(1)
93
94         t(2) = t_0 + dt
95         x(2) = x(1) + v(1) * dt + 0.5 * a * (dt**2)
96         v(2) = v(1) + a * dt
97
98         DO i = 3, n
99             a = -(k / m) * x(i-1)
100             x(i) = 2.0 * x(i-1) - x(i-2) + a * (dt**2)
101             t(i) = t(i-1) + dt
102             v(i-1) = (x(i) - x(i-2)) / (2.0 * dt)
103         END DO
104
105         v(n) = (x(n) - x(n - 1)) / dt
106
107         RETURN
108     END SUBROUTINE VERLET
109
110     SUBROUTINE EULER_PREDICTOR(m,k,t_max,dt,x_0,v_0,t_0,x,v,t)
111         IMPLICIT NONE
112         REAL, INTENT(IN) :: m,k,t_max,dt,x_0,v_0,t_0
113         REAL, ALLOCATABLE, DIMENSION(:), INTENT(OUT) :: x,v,t
114         REAL :: a, x_p, a_p
115         INTEGER :: i,n
116

```

```

117     n = NINT((t_max - t_0) / dt) + 1
118     ALLOCATE(x(n), v(n), t(n))
119
120     x(1)=x_0
121     v(1)=v_0
122     t(1)=t_0
123
124     DO i = 2, n
125         a=-(k/m)*x(i-1)
126         x_p=x(i-1)+v(i-1)*dt+0.5*a*dt**2
127         a_p=-(k/m)*x_p
128         a=0.5*(a_p+a)
129         x(i)=x(i-1)+v(i-1)*dt+0.5*a*dt**2
130         v(i)=v(i-1)+a*dt
131         t(i)=t(i-1)+dt
132     END DO
133
134     RETURN
135 END SUBROUTINE EULER_PREDICTOR
136
137 END PROGRAM EX5

```

5.3 Results

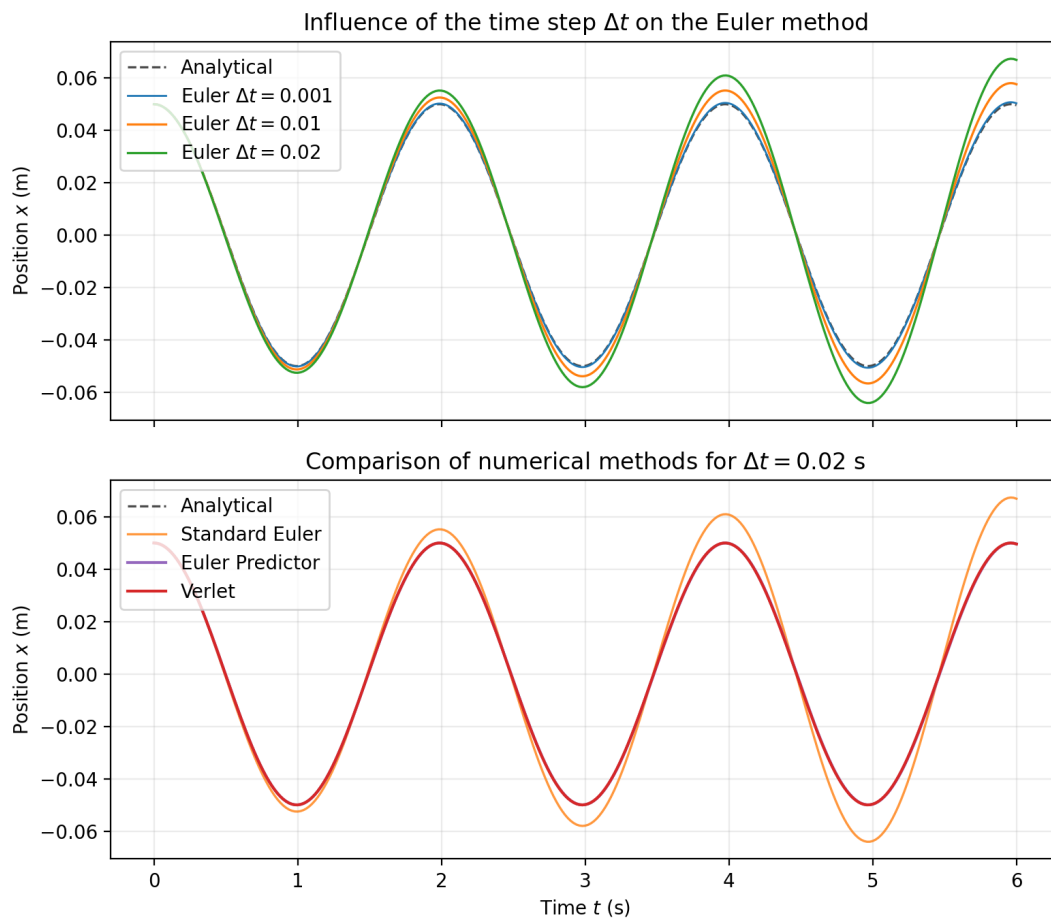


Figure 4: Output of exercise 5

5.4 Discussion & Notes

[Comment on critical or non-trivial components]